AI 创新赛参赛作品 · 产品文档

QQ 待办助手

群消息 → AI 提炼 → 待办看板，全部在你的电脑上完成

产品说明书

产品概述 · 目标用户 · 核心功能 · AI 的角色与价值 · 技术实现思路 · 应用场景
适用版本 v1.0 · 2026 年 10 月

文档类型	产品说明书（面向评审与协作者）
配套文档	《QQ 待办助手 · 产品使用手册》
技术栈	Python 标准库 + requests / 原生前端 / NapCat / DeepSeek
版本日期	2026-10-26



目 录

1 产品概述 4

1.1 它是什么 4

1.2 要解决的真实问题 4

1.3 产品做了什么选择 4

2 目标用户 5

2.1 核心用户画像 5

2.2 使用门槛 5

2.3 明确不服务的场景 5

3 核心功能 6

3.1 功能矩阵 6

3.2 看板 6

3.3 可追溯的来源清单 7

4 AI 的角色与价值 8

4.1 职责边界 8

4.2 时间是怎么说清楚的 8

4.3 同事件判定：怎么做到不重复 9

4.4 AI 失败时怎么办 9

4.5 成本 9

5 技术实现思路 10

5.1 四个角色 12

5.2 一次运行的八个步骤 12

5.3 数据结构 12

5.4 部署与自愈 13

5.5 性能与规模 13

5.6 隐私与安全 13

5.7 可扩展性 14

6 应用场景示例 15

6.1 场景一：一个班的日常（主线场景） 15

6.2 场景二：社团与竞赛信息 15

6.3 场景三：课程群与作业 15

6.4 场景四：小团队与实习兼职信息 15

7 局限与演进路线 **16**

8 交付物与获取方式 **16**

一页读完全文

看什么	一句话结论	详见
要解决的问题	高校信息集中在 QQ 群，漏、复、糊三件事靠人肉记不住	1.2
产品形态	本地优先的单机工具：NapCat 接入 + AI 提炼 + 网页看板	1.1 / 5.1
与同类差别	模型只做语言理解，时间换算与去重交给代码，结论可回溯到原文	4.1 / 4.3
技术骨架	Python 标准库 + 一个 requests；JSON 持久化；计划任务常驻 + 看门狗自愈	5.2 / 5.4
边界与风险	需电脑常开、依赖非官方 NapCat、暂无多人与手机直连	2.3 / 7

说明 本说明书所有截图与示例数据均为**编造内容**，不含任何真实群名、同学昵称与密钥。

1 产品概述

1.1 它是什么

QQ 待办助手是一个**运行在个人电脑上的本地优先工具**：接入你已登录的 QQ，按固定节奏扫描班级群、年级群、社团群的新消息，用大模型判断其中有没有“需要你动手”的要求，再把结果整理成一张可以直接划掉的待办看板。它由三部分组成——消息接入（NapCat）、AI 提炼（DeepSeek）与看板（本地网页），全部在一台机器上闭环运行。

1.2 要解决的真实问题

高校学生的信息分发高度集中在 QQ 群：通知、报名、交表、签到、领教材、竞赛报名、讲座信息混在一起，一天几十到上百条。这里有三个具体痛点：

- **漏**：消息被刷走就再也找不回，重要的截止时间靠“记得”，而不是靠系统。
- **复**：同一份通知被班长、辅导员、任课老师分别转发，人在三个群里各看一遍，心里记三遍。
- **糊**：“今晚9点前”“本周五下班前”“月底前”这类模糊表述，需要自己换算成具体时间，还容易算错。

1.3 产品做了什么选择

选择	理由	代价
本地运行，不上云	群聊内容涉及同学隐私；本地跑也省服务器成本	需要一台常开的电脑
模型只做语言理解，不做计算	让大模型算日期、判重复容易错；代码做这些是确定的	需要一套规则引擎，工程量更大
保留原文与来源	每条待办都能回溯到“哪条消息、谁说的、什么时候说的”	数据结构更重
把配置做成网页向导	手改 JSON 是新手最大的门槛；测试按钮让错误当场暴露	多一个页面要维护
Windows 计划任务常驻	学生用户不会为它开服务器	关机期间抓不到消息

> **说明** 一句话概括产品定位：**一个装在电脑里的“班级群秘书”，把群里散落的要求收拢成一张不会漏的清单。**

2 目标用户

2.1 核心用户画像

画像	典型处境	最痛的点	对本产品的诉求
高校在读学生（主）	同时在上 5~10 个群：班级、年级、课程、社团、竞赛	通知刷屏、截止日期靠记	不漏事、不重复、能一眼看清今天要干嘛
班级/社团干部（次）	既要接收通知，又要向下转发并催进度	重复劳动、统计谁交了	把要求沉淀成清单，自己先不遗漏
小团队协作成员	项目群、实习群、兼职群信息混杂	任务和闲聊混在一起	只保留需要动手的部分

2.2 使用门槛

- 会双击运行一个 .bat / 会复制粘贴一行命令即可；不需要写代码。
- 首次配置全部在网页里完成，Key 与 NapCat 都能当场测试，错误有中文提示。
- 需要理解并接受一个前提：**电脑要开着，QQ 要在线。**
- 需要接受 NapCat 带来的账号风险（见《使用手册》3.2）。

2.3 明确不服务的场景

- 不需要覆盖微信群、钉钉、飞书——本项目只做 QQ 群，把一条链路做深。
- 不做多人协作与任务指派：没有账号体系，看板是本机的单人视图。
- 不做消息回复与自动应答：只读不写，避免误发消息造成社交事故。
- 不做云端 24 小时托管：那是另一种产品形态（需要服务器与合规成本）。

3 核心功能

3.1 功能矩阵

功能	能力边界	当前状态
群消息采集	按群拉取最近 N 条（默认 50）历史，跳过已处理的	已实现
AI 提炼	输出结构化待办（标题、说明、原文时间说法）	已实现
时间解析	中文相对时间 → 绝对时间，标注依据	已实现，单测覆盖
同事件合并	跨群去重、数字冲突不合并、已完成不复活	已实现，单测覆盖
三视图看板	列表 / 时间轴 / 日历，深浅色，手机自适应	已实现
纠错学习	忽略关键词 + 忽略消息 ID 双记忆	已实现
常驻与自愈	开机自启、看门狗、单实例、NapCat 重连	已实现
配置向导	网页填 Key / NapCat，当场测试，立即生效	已实现
多端同步	手机直接访问（需改监听地址）	待开发
多用户协作	账号体系与共享清单	规划中

3.2 看板

看板是整个产品唯一的高频界面，设计目标是“打开三秒就知道今天要做什么”。它不追求信息密度，而是把**紧迫度**变成位置：逾期的在最上面且用红色，今天的有剩余时间提示，没写截止的单独成组放在最后，不与有期限的事混在一起制造焦虑。

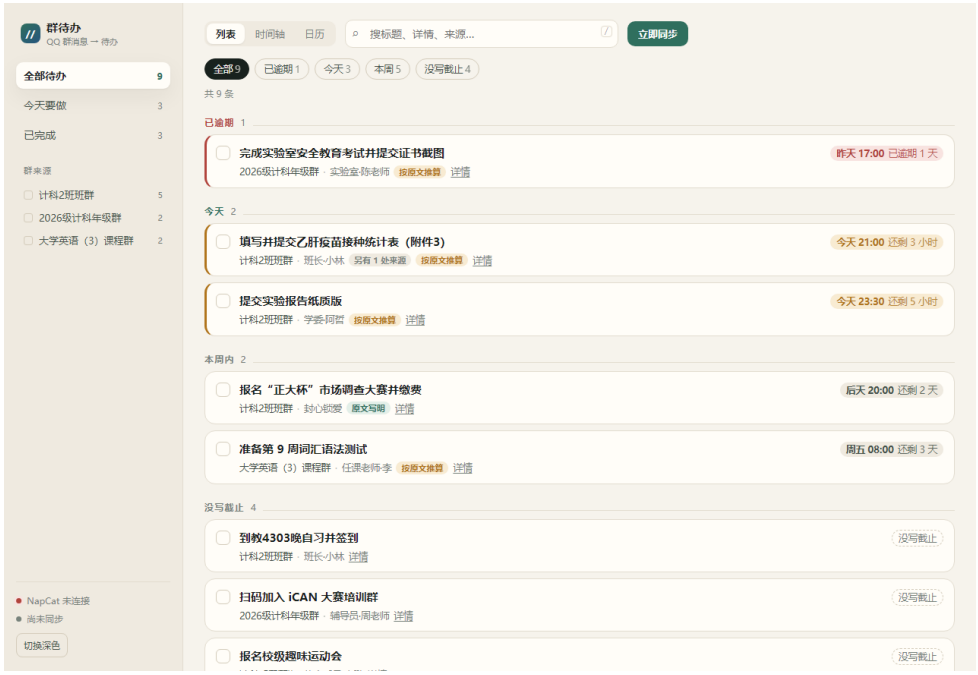


图 1 列表视图：按紧迫度分组，右侧给出剩余时间或“没写截止”

3.3 可追溯的来源清单

每条待办都带 sources 数组。同一份通知被三个群转发时，卡片上显示“另有 2 处来源”，展开可见每个群的名称、发送者、发送时间与原文引文，以及该来源是原文（exact）还是转发匹配（matched）。这是产品与“AI 随口总结”最重要的区别：**结论永远能回到证据。**

4 AI 的角色与价值

这一章说明“AI 到底干什么、为什么这样分工”，也是本项目在准确率与成本上的核心设计。

4.1 职责边界

环节	由谁做	为什么
判断“这段话里有没有要我做的事”	AI	语义判断，规则写不全：同一句“记得看群文件”在不同语境下含义不同
抽取标题与说明	AI	概括与改写，人读起来才顺
把“今晚9点前”照抄出来	AI	只做识别，不做换算，避免它算错日期
把“今晚9点前”换算成 2026-10-26 21:00	代码	确定性算法，可单测，跨月跨周不会错
判断两条消息是不是同一件事	代码	数字比对与字面重合度可解释；交给模型会时对时错且无法复现
去重、幂等、已完成不复活	代码	状态管理必须是确定的

> **说明** 设计原则可以概括成一句：**让模型做它擅长的模糊判断，让代码做它擅长的精确计算。**把日期换算交给模型，是同类产品最常见的错误来源。

4.2 时间是怎么说清楚的

模型只输出原文说法（deadline_raw），代码按消息发送时间做换算，并标注来源等级。以下是真实规则示例：

原文说法	消息时间	换算结果	标注
今晚9点前	10-26 15:10	10-26 21:00	inferred（按原文推算）
明天下午6点前	10-26 15:10	10-27 18:00	inferred
本周五下午6点前	10-26 15:10	10-30 18:00	inferred
下周五	10-26	11-06 23:59	inferred
10月9日12:00前	10-06	10-09 12:00	explicit（原文写明）
月底前	10-26	10-31 23:59	inferred
（没提时间）	—	null	none（原文没写截止）

看板上“按原文推算”的小标签就是 inferred；“没写截止”意味着原文真的没有时间，而不是抓取失败——系统不会拿消息发送时间冒充截止时间。

4.3 同事件判定：怎么做到不重复

1. **标题归一**：先去掉“通知 / 请 / 提交 / 相关”这类通用词，再比字面重合度，避免“提交乙肝表”和“填写乙肝表”各存一条。
2. **数字必须一致**：**10月9日** 与 **10月11日**、附件2 与附件3 一律视为两件事——宁可重复，不可误合并。
3. **同批转发补挂来源**：同一轮抓到的相似消息不新开卡片，而是把来源补到已有条目的 sources 里，标为 matched。
4. **已完成不复活**：已完成条目只要截止日相同，不会因为别的群转发又变成未完成。
5. **换期限算新任务**：同名任务换了截止日期（比如下周的新表），正常作为新条目出现。

4.4 AI 失败时怎么办

- 模型返回的不是合法 JSON：**丢弃本次结果**，把消息记为已处理并写日志，不污染待办数据，也不反复重试烧钱。
- Key 缺失或失效：看板顶部显示失败原因，并引导到配置向导；历史待办仍然可读。
- 限流 429：记录错误、下一轮再试；已处理消息不会重复发送。
- NapCat 掉线：整轮跳过，不空转、不产生错误数据。

4.5 成本

模型只在“本轮确实有新消息”时才被调用；没有新消息时零调用、零费用。按默认节奏（每 3 分钟一次）与中等活跃度（每天约 300 条新消息）估算：

项目	估算口径	数值
输入 token	每天约 1.5 万，月 45 万	约 0.9 元/月
输出 token	每天约 2 千，月 6 万	约 0.5 元/月
合计	按 deepseek-chat 官方价目估算	约 1.4 元/月，一年约 17 元

注：价格为按公开价目做的估算，实际以平台账单为准；本项目无服务器成本，其余开销只有你自己的电脑电费。

5 技术实现思路

整套系统是“客户端—服务器—HTTP + JSON + 持久化”的一个完整小闭环，架构与运行流水线见下图：



图 2 整体架构（四个角色）与一次运行的流水线（八个步骤）

5.1 四个角色

角色	职责	技术选型
QQ + NapCat	持有登录态，把群消息暴露成 HTTP 接口	NapCat，OneBot v11 协议
main.py（中枢）	既是客户端（向 NapCat、DeepSeek 要数据），也是服务器（给看板提供页面与接口）	Python 标准库 + requests，无框架
DeepSeek	语义理解，返回结构化待办	OpenAI 兼容的 chat/completions
看板 index.html	展示与操作	单文件 HTML + 原生 JS，无构建、无依赖

5.2 一次运行的八个步骤

步骤	做什么	关键点
1 拉消息	get_group_list 拿群列表，get_group_msg_history 拉最近 N 条	只处理配置允许的群
2 转纯文本	把分段消息（文本/图片/文件）拼成一句话	图片记为 [图片]，文件记为 [文件名字]
3 去重	按消息 ID 跳过已处理、按关键词跳过已教过的	幂等：同一消息永不重复处理
4 交 AI 提取	只发新消息，要求返回 JSON	无新消息则不调用
5 合并清洗	同事件判定、补挂来源、时间换算、字段规范化	代码负责，可单测
6 记录已处理	把本批消息 ID 写入 seen.json	下次不重复、不重复收费
7 起看板	http.server 提供页面与 /api 接口	单实例、后台常驻
8 人操作	点完成/删除/忽略 → POST → 改 JSON	操作即时落盘

5.3 数据结构

不用数据库，用 JSON 文件。理由：单用户、数据量小（一年几千条）、要能直接打开看、要能整包备份迁移。写入时先写临时文件再替换，并保留 .bak，避免断电写坏。

```
{
  "title": "填写并提交乙肝疫苗接种统计表（附件3）",
  "published_at": "2026-10-26 15:10",
  "start_at": null,
  "deadline": "2026-10-26 21:00",
  "deadline_basis": "inferred",       // explicit / inferred / none
```

```
  "deadline_raw": "今晚9点前",        // 原文，可追溯
  "event_key": "a1b2c3d4e5f6",       // 同事件指纹
  "sources": [
   {"group": "计科2班班群", "sender": "班长·小林", "msg_id": "101",
    "time": "2026-10-26 15:10", "quote": "……", "confidence": "exact"},
   {"group": "2026级计科年级群", "sender": "辅导员·周老师", "msg_id": "102",
    "time": "2026-10-26 15:25", "quote": "……", "confidence": "matched"}
  ],
  "done": false
 }
```

5.4 部署与自愈

机制	实现	解决什么
开机自启	计划任务 QQTodoBot：登录桌面时启动 pythonw main.py --background	不用每次手动开
看门狗	QQTodoBot-Watchdog 每 2 分钟探测端口，无响应则重拉主任务	进程被强杀后自动恢复（实测约 40 秒）
NapCat 守护	连接失败按退避重试，必要时用 napcat_qq_uin 快速登录拉起 NapCat	免扫码、掉线自愈
单实例	端口占用检测 + 进程互斥	重复启动不会开出两个进程抢数据
日志滚动	logs/app.log 超过 2MB 滚动为 .1	长期运行不撑爆磁盘

5.5 性能与规模

- 单轮处理量 = 群数 × 50 条上限，落在一次模型调用的合理窗口内；默认 3 分钟一轮，对 NapCat 与本机压力可以忽略。
- 看板是静态 HTML + 一次 /api/todos 拉取，千级条目在浏览器里仍是毫秒级渲染。
- 数据以 JSON 全量读写：一年几千条仍是百 KB 量级，暂无引入数据库的必要；若条目上万，可按月分文件或换 SQLite。

5.6 隐私与安全

- 全部数据在本机；不做任何上传、不采集设备信息。
- 只有“提炼”这一步把消息文本发给 DeepSeek；可用 target_groups 限定只处理必要的群。
- Key 只存在本地 config.json，并已加入 .gitignore；界面回显只露头尾，回显值不会覆盖真实 Key。
- 看板只监听 127.0.0.1，同一局域网的其他设备访问不到（这也是手机暂时不能直接访问的原因）。

- 仓库里的截图与演示数据全部为编造内容，不含真实群名与同学昵称。

5.7 可扩展性

- **换模型**：改 deepseek_base_url 与 deepseek_model 即可接任何 OpenAI 兼容接口。
- **换前端**：所有能力都在 /api 上，前端可以整体替换（已有 /api/todos、/api/todo/done|undo|delete|ignore、/api/sync、/api/config）。
- **换数据源**：NapCat 之后接任何 OneBot v11 实现都可以复用同一套采集逻辑。
- **加提醒**：截断点清楚——拿到 todo 后可以推送微信/邮件/桌面通知，不需要改动提炼层。

6 应用场景示例

6.1 场景一：一个班的日常（主线场景）

输入	一天里班群 40 条、年级群 15 条、英语课群 8 条。其中三条是要求：今晚 9 点前交乙肝统计表、今晚 11 点半前交实验报告纸质版、10 月 28 日 20:00 前报名市场调查大赛。
系统的处理	把“今晚9点前”换算成今天 21:00 并标注为推算；把年级群转发的那份统计表合并进同一条，标为“另有 1 处来源”；把 40 条里的“收到”“谢谢老师”全部丢掉。
输出与价值	看板顶部三条，其中两条带剩余小时数，日历上 26 日与 28 日各有一个点；从“翻群 + 记时间”变成“打开看一眼”，漏交的概率显著下降。

6.2 场景二：社团与竞赛信息

输入	竞赛群、培训群、学院通知里混着报名截止、扫码进群、培训时间。
系统的处理	报名类带明确截止时间，进“本周内”；“扫码加入培训群”这类没有截止时间的，进“没写截止”分组，不会被当成紧急事项挤占视线，但也不会被丢掉。
价值	竞赛信息最容易“看到了但错过截止”。把截止日期变成日历上的一个点，是这类信息最需要的形态。

6.3 场景三：课程群与作业

输入	任课老师发“下周一上课时做词汇语法测试”“实验报告电子版发课程邮箱”。
系统的处理	区分“有明确期限”和“只说了要做”：有期限的进本周，没期限的进“没写截止”，避免所有作业都堆在“今天”制造焦虑。
价值	把“模糊的长期要求”和“本周必须交的东西”分开放，人的注意力才能用在对的地方。

6.4 场景四：小团队与实习兼职信息

输入	项目群里的任务分派、兼职群里的报名方式、实习群里的投递截止。
系统的处理	用 target_groups 只处理这几个群，把生活群、家人群彻底排除在外，既不打扰也不越界。
价值	同一套工具，靠“选群”而不是“选功能”适配不同人群，这是它能从校园延伸到职场的原因。

7 局限与演进路线

当前局限	影响	演进方向
必须有一台开机的 Windows 电脑	关机期间消息抓不到，只能补拉历史	提供 Linux/Docker 部署，或轻量云托管
只监听本机地址	手机无法直接访问	增加“允许局域网访问”开关与访问口令
单个用户、无账号	无法多人共享同一份清单	账号体系 + 共享清单 + 指派
只读 QQ 群	不能回消息、不能催办	按需增加“一键提醒自己”而不做自动回复，避免误发
忽略规则基于关键词	同义不同词仍可能重复提醒	引入向量相似度做语义级忽略
依赖 NapCat	存在平台风控风险	抽象数据源接口，支持官方机器人渠道

8 交付物与获取方式

内容	位置
源代码（Python + 单文件前端）	仓库根目录 main.py / web/index.html
一键安装脚本与看门狗	install.ps1 / uninstall.ps1 / watchdog.ps1 / start.bat
配置向导	web/setup.html，运行时访问 /setup
单元测试	tests/test_core.py（时间换算、同事件判定、合并落库）
演示数据与截图	docs/demo-data / docs/screenshots（全部为编造内容）
使用手册（本文档的姊妹篇）	docs/手册与说明书/QQ待办助手-使用手册.pdf

运行环境：Windows 10/11 + Python 3.9 以上 + NapCat + 一个 DeepSeek Key。除 DeepSeek 的 token 费用外无其他开销。